Assignment 6

The Adpative Resonance Theory Neural Network

Connor Prikkel

ECE 550: Artificial Neural Networks

09 November 2024

University of Dayton

Contents

1 Introduction

In this homework, the student explores and implements an Adaptive Resonance Theory (ART) neural network for the recognition of handwritten digits. This network utilizes unsupervised learning, or the method of learning patterns and classifying output without labels.

2 Methodology

2.1 Background

Unsupervised learning is particularly useful when there is ambiguity in the data being tested, as it provides a concrete way of defining when an output is similar enough to pre-existing classes and should be labeled as such versus being defined as its own new class. The ART network accomplishes this by testing new its new patterns against each existing one, represented by class vector C. An example two layer ART network containing inputs x1, x2, x3, and output classes C1, C2, C3 is given below in Figure 1.

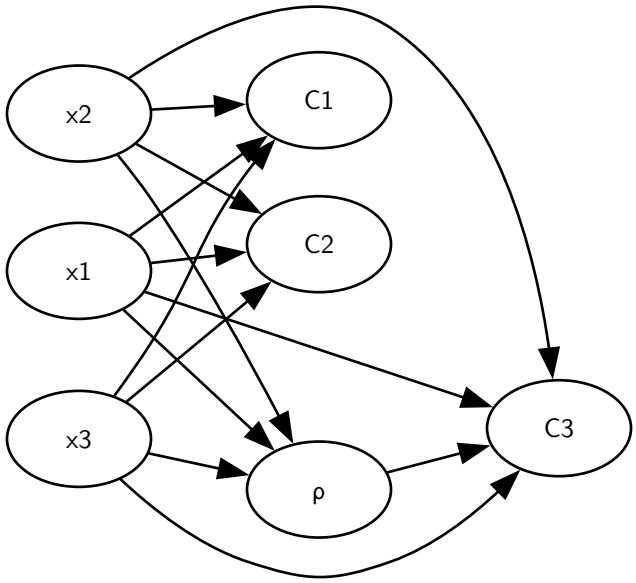


Figure 1: Example ART Network

The inputs x1, x2, ... xN make up the comparison layer and are mapped to the recognition layer containing all the output classes through weight matrix w_{ji}. 'ρ' is known as the vigilance parameter. It ensures new patterns can be learned and that existing classes do not deteriorate with the addition of new patterns.

In order to create such a network, choice function CF_j is defined to determine what class a pattern belongs to based on its distance to already existing patterns. The equation for it is given below:

$$\text{CF}_j = \frac{\sum_{i=1}^{N} x_i \wedge w_{\text{ji}}}{\alpha + \sum_{i=1}^{N} w_{\text{ji}}} \tag{1}$$

where x_i is each input data sample of each pattern s = 0, 1, ..., 9, N is the total number of input samples, α is the choice parameter, and $\wedge$ is the fuzzy AND, which takes the minimum value of x_i and w_{ji}. This function is computed for all output classes before the 'winner node', CF_J, is chosen by Equation 2:

$$\text{CF}_J = \text{Argmax}\{\text{CF}_j\} \tag{2}$$

Next, the vigilance, VF_J, is computed by:

$$\text{VF}_J = \frac{\sum_{i=1}^{N} x_i \wedge w_{\text{ji}}}{\sum_{i=1}^{N} x_i} \tag{3}$$

The vigilance is then compared to the vigilance parameter: if $\mathrm{VF}_J < \rho$, the current pattern is not similar enough to the current CF_J and the test fails. CF_J is then set to 0 and the next winner node is computed by Equation 1. However, if $\mathrm{VF}_J \geq \rho$, the test passes. The weights are then updated by the equation below:

$$w_{\mathrm{ji}}{}^{\mathrm{New}} = w_{\mathrm{ji}} \wedge x_i \tag{4}$$

If all existing nodes fail the vigilance test, a new output class is created and the weights are updated by Equation 5:

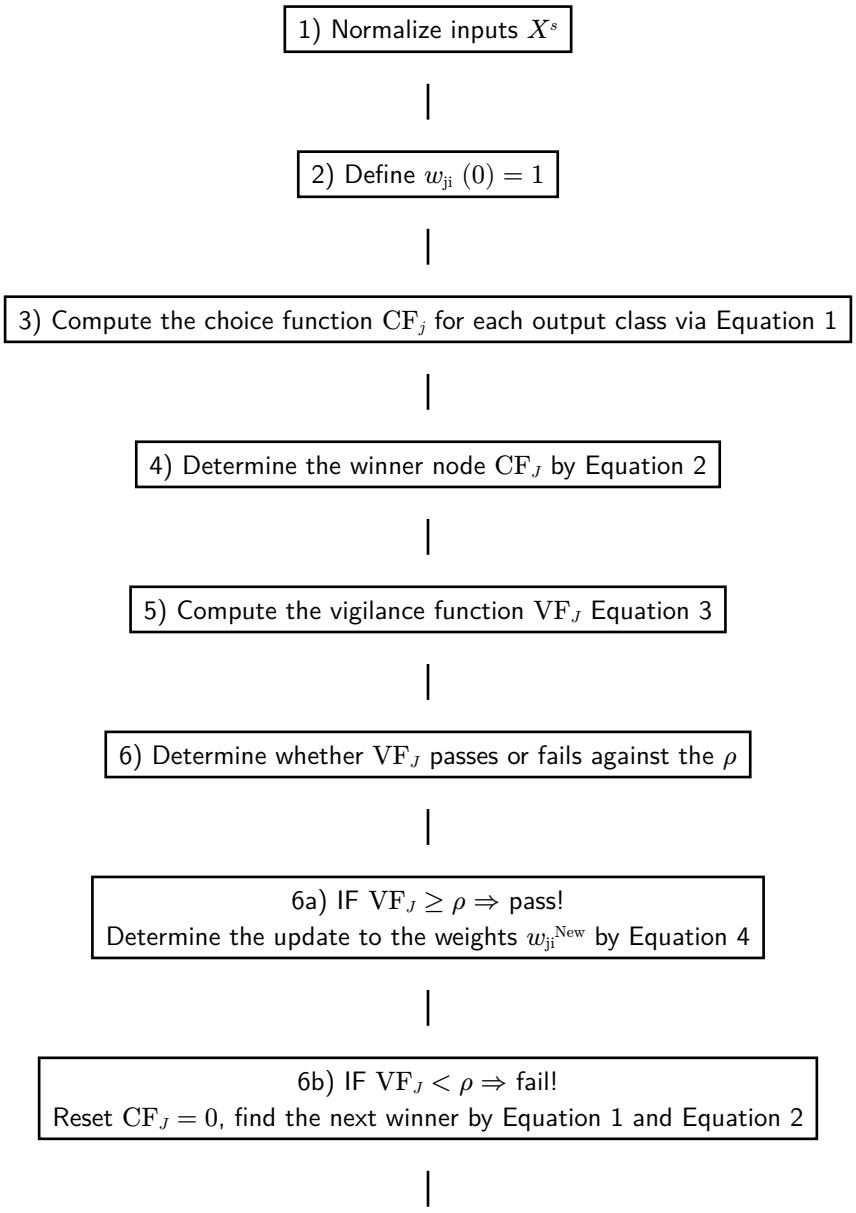
$$w_{\mathrm{ji}}{}^{\mathrm{New}} = \beta w_{\mathrm{ji}} \wedge x_i + (1-\beta) w_{\mathrm{ji}} \tag{5}$$

where β is the learning rate parameter. These are repeated for all patterns and until the number of output classes stabilizes. There is a preprocessing technique known as complement coding that can greatly aid with this pattern classification process. It involves concatenating each input sample, X^s, with its complement, $!X^s$ described by Equation 6 below.

$$X^{\mathrm{sc}} = X^s(!X^s) \tag{6}$$

2.2 Program Flowchart

The following 9 steps define the training and testing process of the ART neural network.



7) IF all nodes fail the vigilance test, create a new output class and update the weights by Equation 5

|

8) Repeat Steps (3) - (7) for each input pattern s = 0, 1, ... 9

|

9) Repeat Steps (3) - (8) until stabilized, i.e. no new classes created

3 Results

3.1 Procedure

Handwritten digits containing numbers 0 - 9 from the MNIST dataset are loaded and partitioned into 200 training images and 100 testing images. The ART network is then implemented in Python code; it is initialized and trained first with no preprocessing and then with complement coding. It is then tested on both the training and testing data for these two training scenarios.

3.2 Visualization and Evaluation

Plots containing the percentage error for recognition on both the training and testing data for a variety of learning rates and vigilance parameter values are given below in Figure 2 and Figure 3.

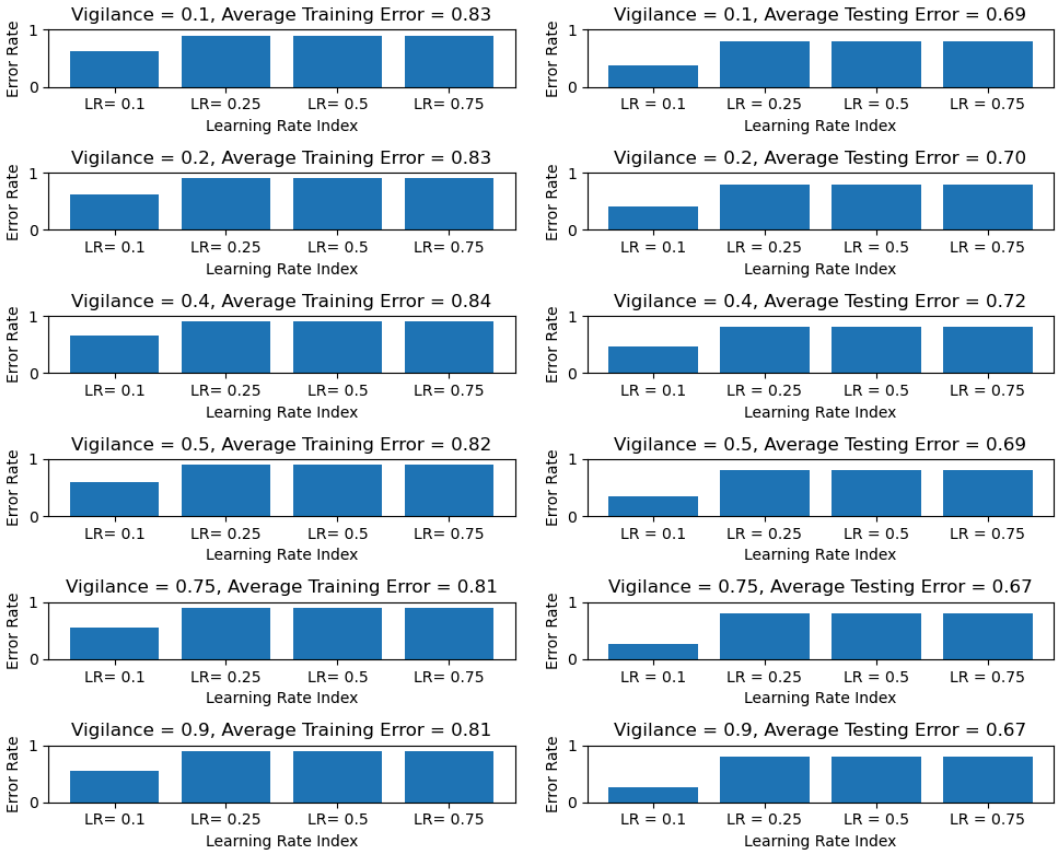


Figure 2: Percentage Error, No Complement Coding

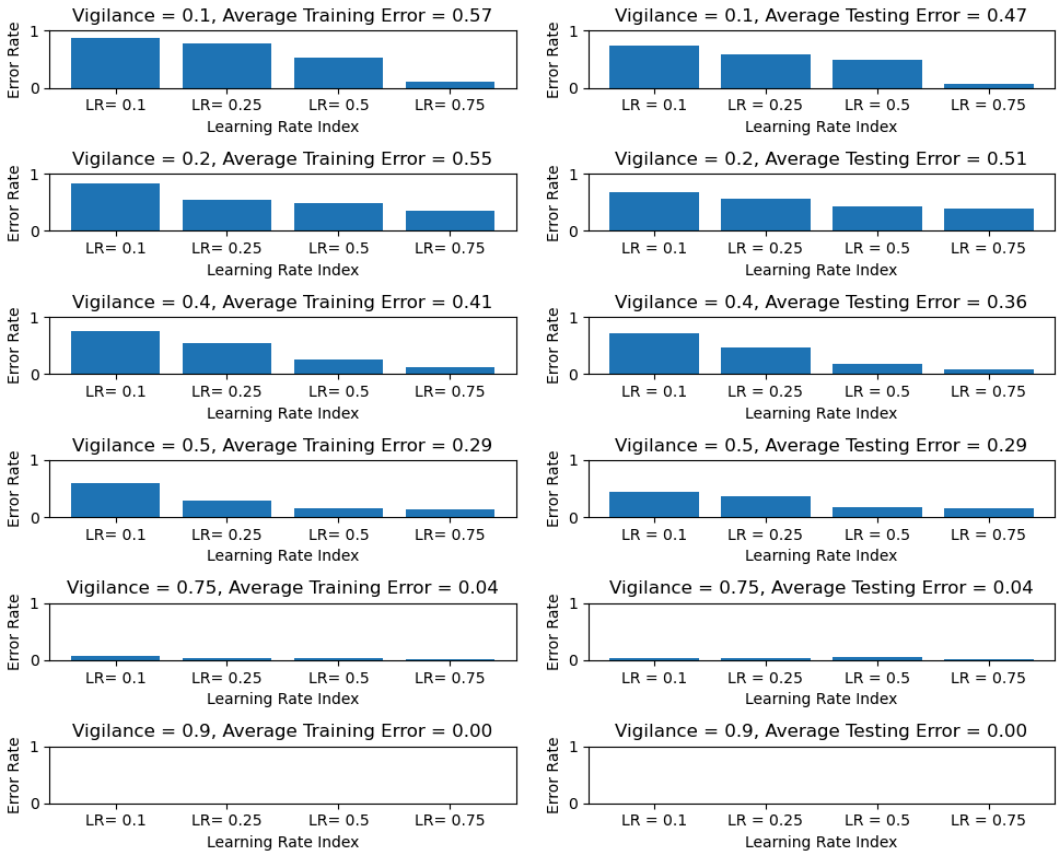


Figure 3: Percentage Error, Complement Coding

Figure 2 reveals that utilizing different learning rate and vigilance values did not significantly impact performance on the ART network trained without complement coding. This network consistently performed fairly poor except for when it is trained with a learning rate of 0.1. Although it is encouraging that the percentage error is slightly lower for the test data, the overall large error rates for this network likely indicate that it struggled to create distinct, separable classes. Increasing the number of samples per digit that the network trains on is a possible solution to improve performance and better ensure the of output classes.

Figure 3 illustrates the effectiveness of complement coding for allowing the network to better distinguish between patterns by more fully representing each input in a 2D vector space. The average training and testing error decreases as learning rate and the vigilance parameter increase. This is expected as a higher vigilance parameter indicates the network is more selective and less prone to misclassifying an output class, whereas a higher learning rate allows the network to adapt to new patterns more quickly.

4 Conclusion

The ART network and its learning mechanism are investigated and implemented in Python code. The student learned of the advantage of an unsupervised learning network in its ability to derive and find connecting features between distinct output classes. This advantage becomes even more clear with other more practical real world applications where the data being trained and tested might not be easily separated as numerical digits. However, this type of network has several tradeoffs. For one, it requires careful manipulation of multiple parameters including β and ρ to ensure it reaches stabilization and effectively separates the data into meaningful and distinct classes. It is more prone to overfitting and underfitting without proper training as revealed by the terrible performance on the network without complement coding. Therefore, it should be utilized with cautious optimism.